News Reporter

Memoria técnica · Agente automatizado de generación de noticias con IA

PROYECTO ACADÉMICO • N8N 2.31 • OCTUBRE DE 2026

1. Objetivo del proyecto

Desarrollé News Reporter, un agente de generación de contenidos utilizando n8n, con el objetivo de automatizar la obtención, el procesamiento y la publicación de noticias tecnológicas.

Busqué construir una solución funcional, que pudiera ejecutarse con herramientas gratuitas y que fuera relativamente sencilla de mantener y de ampliar.

2. Tecnologías utilizadas

Para desarrollar el proyecto elegí las siguientes herramientas:

- n8n 2.31 y Docker: utilicé n8n como plataforma de automatización, ejecutándolo localmente mediante Docker.
- RSS de El País: seleccioné su sección de Tecnología como fuente de noticias reales en español.
- Google Gemini 3.5 Flash-Lite: lo integré para redactar los resúmenes y generar los prompts para las imágenes.
- Pollinations AI — Flux Schnell: utilicé este servicio para generar imágenes mediante créditos gratuitos.
- Telegram Bot API: creé un bot para recibir las publicaciones generadas por el agente.
- Data Tables de n8n: incorporé una tabla para registrar las noticias publicadas y evitar duplicados.

Uno de mis objetivos principales fue desarrollar todo el proyecto sin gastar dinero. Para conseguirlo, aproveché las cuotas gratuitas de los servicios y los créditos promocionales disponibles en Pollinations.

3. Funcionamiento del agente

El workflow comienza con un disparador manual. Elegí esta modalidad para poder controlar las ejecuciones mientras desarrollaba y probaba el proyecto.

Primero, utilizo un nodo HTTP Request para consultar el RSS de Tecnología de El País y recuperar las noticias en formato XML.

A continuación, convierto esa información a JSON, separo las noticias en elementos individuales y normalizo sus fechas de publicación. Después las ordeno cronológicamente y selecciono la más reciente.

Antes de enviarla a la inteligencia artificial, consulto una Data Table para comprobar si esa noticia ya fue publicada. Si el enlace está registrado, el workflow se detiene.

Si se trata de una noticia nueva, envío a Gemini su título, descripción, fecha y fuente. El modelo genera un resumen en español, con un tono informativo y cercano, y también un prompt en inglés para generar una imagen relacionada.

Luego utilizo un nodo Code con JavaScript para transformar la respuesta de Gemini en un objeto JSON y validar que contenga los campos necesarios.

Con el prompt visual generado por Gemini, realizo una petición HTTP a Pollinations para crear una imagen mediante el modelo Flux Schnell.

Finalmente, preparo el mensaje con el título, el resumen y el enlace original de la noticia, y lo envío a Telegram junto con la imagen.

Después de recibir la confirmación del envío, registro la publicación en Data Table utilizando la operación Upsert.

4. Decisiones técnicas y editoriales

Durante el desarrollo tomé varias decisiones buscando que el agente fuera funcional, fiable y económico.

En primer lugar, elegí utilizar un disparador manual para evitar ejecuciones involuntarias mientras realizaba pruebas.

También decidí procesar una sola noticia por ejecución. De esta manera reduzco el consumo de las APIs y evito generar múltiples imágenes innecesariamente.

Otro aspecto importante fue la calidad de la información. Como el RSS generalmente proporciona solamente el título y un extracto, configuré Gemini para que trabajara exclusivamente con esos datos, sin inventar información ni asumir que había leído el artículo completo.

Además, decidí conservar el enlace original de la noticia durante todo el proceso, para que cualquier persona pueda consultar la fuente.

En cuanto a las imágenes, inicialmente observé que el generador podía crear marcas comerciales ficticias cuando intentaba representar productos de empresas conocidas. Para solucionar este problema, mejoré las instrucciones del prompt y prioricé ilustraciones conceptuales sin logotipos, carteles ni marcas inventadas.

5. Prevención de duplicados y gestión de errores

Una de las mejoras más importantes que incorporé fue el control de noticias duplicadas.

Inicialmente probé un nodo Remove Duplicates, pero detecté un inconveniente: podía registrar una noticia como procesada aunque posteriormente fallara su publicación en Telegram.

Para resolverlo, sustituí ese mecanismo por una Data Table que almacena las noticias publicadas.

Actualmente, antes de procesar una noticia, consulto la tabla utilizando su enlace como identificador. Si ya existe, detengo el flujo antes de llamar a los servicios de inteligencia artificial.

Por otra parte, configuré el registro para que se realice después de la publicación en Telegram. Utilicé Upsert para reducir el riesgo de filas duplicadas y agregué reintentos ante posibles errores transitorios.

También incorporé validaciones en JavaScript para detectar respuestas vacías, formatos JSON incorrectos o campos faltantes en la respuesta de Gemini.

Aun así, reconozco una limitación: si Telegram recibe una publicación, pero n8n no obtiene la confirmación correspondiente, podría producirse un duplicado al reintentar la ejecución. En ese caso, sería necesario revisar el chat antes de repetir la publicación.

6. Pruebas realizadas

Durante el desarrollo fui probando cada componente por separado antes de integrarlo al workflow completo.

Primero comprobé que podía obtener noticias reales desde el RSS y convertir correctamente la información XML en JSON.

Después verifiqué la conexión con Gemini, la generación del resumen y del prompt visual, y la transformación de su respuesta en campos estructurados.

También conseguí generar imágenes desde n8n utilizando Pollinations y publicar el resultado en Telegram junto con el resumen y el enlace original.

Por último, comprobé que Data Table podía registrar una noticia publicada y que una nueva consulta bloqueaba correctamente el mismo enlace.

La configuración final de Upsert y el identificador del mensaje de Telegram también quedaron incorporados al workflow exportado.

7. Requisitos para importar el workflow

Exporté el workflow a un archivo JSON para facilitar su entrega y permitir que pueda importarse en otra instalación compatible de n8n.

Por seguridad, eliminé el Chat ID personal de la copia de entrega y no incluí las claves API ni el token del bot.

Para reproducir el proyecto en otro entorno, es necesario:

1. Configurar las credenciales de Gemini, Pollinations y Telegram.
2. Crear una Data Table llamada news_reporter_publicadas con las columnas enlace (String), titulo (String), fecha_publicacion (Date/Time) y telegram_message_id (Number).
3. Seleccionar esa tabla en los dos nodos Data Table del workflow.
4. Reemplazar REEMPLAZAR_CHAT_ID por el identificador del chat de Telegram correspondiente.
5. Verificar la disponibilidad de los modelos y los créditos gratuitos antes de realizar una ejecución.

Estos pasos son necesarios porque el archivo JSON contiene la configuración del workflow, pero no las credenciales secretas ni los registros almacenados en la Data Table original.

8. Limitaciones y mejoras futuras

Si bien conseguí desarrollar un agente funcional utilizando herramientas gratuitas, reconozco que existen algunas limitaciones.

La principal es la dependencia de servicios externos y de sus cuotas gratuitas. Además, el RSS no siempre proporciona suficiente información para elaborar resúmenes detallados.

En el futuro, me gustaría incorporar un disparador programado para que el agente pueda ejecutarse automáticamente a intervalos definidos.

También considero interesante mejorar la gestión de errores y permitir que el agente seleccione otra noticia cuando la más reciente ya haya sido publicada, en lugar de detenerse.

Estas mejoras permitirían evolucionar News Reporter hacia un sistema de publicación más autónomo, manteniendo la calidad de la información y el control sobre los recursos utilizados.